

L6 CSK Decoder Update

Date: 2026-07-06

Summary

The QZSS L6D/E CSK decoder in `src/sdr_ch.c` was rearchitected. Symbol detection and code tracking are now decoupled:

1. The CSK correlation-peak search runs on **chip-binned data with a small fixed-size FFT** ($M = 10800$), instead of a full sample-rate FFT pair per symbol. Its cost no longer depends on the sampling rate.
2. The E/P/L correlations are produced by the **standard time-domain correlator** against the CSK-shifted code, instead of interpolating the FFT correlation. This removes the correlation-peak smearing that degraded L6 pseudoranges.
3. After L6 frame sync, the peak search is **narrowed from 511 to 256 candidate shifts**, eliminating the ± 256 -chip alias candidates.

Measured on the `sdr_ch_update()` benchmark (one L6D channel, one 4 ms update), the new decoder is 3-8x faster; at 24 Msps on a Raspberry Pi 5 the per-channel cost drops from 2534 to 518 us (FFTW + wisdom) so that 8 L6 channels fit in about 26 % of the 4 cores. Tracking robustness on live data improved substantially (see Verification).

Previous Implementation and Problems

Per 4 ms symbol, the previous decoder computed a full circular correlation at sample resolution: carrier wipeoff over $N = f_s \times 4$ ms samples, forward FFT of size N , complex multiply with one of 10 fractional-delay code-FFT banks, and backward FFT of size N . `CSK()` then searched the ± 255 -chip lag window for the peak and produced the E/P/L correlator outputs by *linear interpolation* of the sample-spaced correlation.

- **Cost:** two N -point FFTs per symbol per channel, N proportional to f_s . At 24 Msps ($N = 96000$) one channel consumed 2.5 ms of a 4 ms cycle on a Pi 5 even with FFTW and MEASURE wisdom, limiting a Pi 5 to about one L6 channel per core.
- **Pseudorange quality:** the E/P/L values interpolated from the sample-spaced FFT correlation flatten the correlation peak, biasing the DLL discriminator and producing large pseudorange errors.
- **Memory:** the 10 fractional code-FFT banks occupy $10 \times N \times 8$ bytes per channel (7.7 MB at 24 Msps).

New Decoder

Signal structure

The L6 code is 10230 chips (2.5575 Mcps) per 4 ms symbol. On air, L6D and L6E are time-division multiplexed at twice the chip rate: each code chip consists of two 5.115 MHz sub-chip slots, L6D occupying the even slot and L6E the odd slot (the stored code of `sdr_gen_code()` is the 20460-slot sequence with zeros in the other signal's slots). CSK shifts the code cyclically by the 8-bit symbol value in chip units.

Chip binning (`sdr_bin_csk()`)

Carrier-wiped IF samples are accumulated chip by chip over the signal's TDM slot only:

```
spc = N / len_code ; samples per TDM slot
t(k) = coff + slot * spc + k * 2 * spc ; start of chip k's slot (samples)
bins[k] = sum of IQ[j], j in [ceil(t(k)), ceil(t(k) + spc))
```

`coff` is the fractional code offset in samples, so sub-sample code alignment is applied continuously at the bin boundaries (the 10-bank fractional quantization is no longer needed). `slot` selects L6D (0) or L6E (1). The loop is plain scalar code; SIMD versions (AVX2/NEON) were implemented and benchmarked, but the measured gain was at most 8 % of the channel cost at 24 Msps and zero at 12/16 Msps, so they were dropped to keep the code simple. There is no sampling-rate limit.

Chip-domain FFT correlation

The circular correlation over all CSK shifts is computed on the 10230-chip sequence with a fixed FFT size $M = 10800$ ($2^4 \times 3^3 \times 5^2$), using the code **periodically extended** by $W = 280$ chips on both sides and zero-padded:

```
L = 10230, W = 280, M = 10800 >= L + 2W
e[m] = c[(m - W) mod L], m in [0, L + 2W) ; extended code
corr = IFFT(FFT(bins_padded) * conj(FFT(e_padded))) / M^2
X[s] = corr[M - W + s], s in [-W, W] ; correlation at shift s
```

For s in $[-W, W]$ every data chip n in $[0, L]$ maps to the extension index $n - s + W$ in $[0, L + 2W)$, so $X[s]$ equals the exact circular correlation with no wrap-around aliasing as long as $M \geq L + 2W$. The peak of $|X[s]|^2$ over the search window gives the CSK shift and the symbol `sym = 255 - s mod 256` as before.

Because M is fixed, the FFT stage cost is independent of f_s ; only carrier mixing, binning, and the E/P/L correlator scale with the sampling rate. The per-channel code data shrinks from the 10 code-FFT banks to one M -point code FFT (84 KB) plus the ordinary int8 code bank shared with the standard correlator path.

E/P/L correlations

The correlator outputs are produced by the fused standard correlator with the detected CSK shift added to the code offset:

```
R = N / 10230 ; samples per chip
sdr_corr_std(buff, ix + i, N, fs, fc, phi + fc * i / fs, code,
             coff * fs - i + csk * R, pos, npos, 1, C, C1)
```

This yields true (non-interpolated) E/P/L correlations of the CSK-shifted code, with the same sharpness as any BPSK signal at the same sampling rate, fixing the pseudorange degradation. The known CSK shift is added as a deterministic offset, so the DLL still estimates the true code phase.

The `pol = 1` argument is essential. `sdr_corr_std()` splits the correlation at the code wrap-around j into two partial sums and normally detects a data-bit polarity flip from their dot product. For L6 the wrap position moves with the CSK shift each symbol; whenever one partial sum covers only a few samples it is noise-dominated (at typical C/N0 the per-sample SNR is about $1e-3$) and the flip detection degenerates to a coin toss, negating the whole correlator output for roughly 8 % of symbols. On live data this appeared as random 180-degree IP flips, repeated PLL kicks, and loss of lock on weaker satellites. Since the L6 correlation window is symbol-aligned and circular, no polarity flip can occur, and `pol = 1` forces a fixed polarity.

Peak-search narrowing after frame sync

Before frame sync the code-phase reference established at acquisition is offset by the (unknown) symbol transmitted during acquisition, so the search must cover shifts s in $[-255, +255]$. This window contains each symbol value twice (s and $s - 256$ give the same symbol); picking the wrong alias still decodes the correct symbol but places the E/P/L correlator 256 chips off, wasting the symbol and perturbing the loops.

`decode_L6_frame()` synchronizes the frame on preamble *differences* and recovers the constant symbol offset `off` (stored as `nav->coff = (off - 1) * T / 10230`). Once `nav->fsync` is set, the true symbol is `s_true = (sym + off) mod 256` and the detected shifts obey

```
ix = ref - s_true, ref = const in [0, 255]
```

`CSK()` anchors `trk->csk_ref = ix + (sym + off) mod 256` once per frame lock and then searches only the 256 shifts $[ref - 255, ref]$. The anchor is cleared whenever frame sync is lost (RS failure), falling back to the full ± 255 search, so a wrong anchor self-heals within one frame.

API and Data-Structure Changes

- `sdr_corr_std()`: new `pol` argument (0: detect polarity flip at the code wrap-around, 1: fixed polarity). All existing callers pass 0.

- `sdr_bin_csk()` : new API (chip binning, see above).
- `sdr_corr_fft()` : carrier mixing fused into the API (`buff`, `ix`, `fs`, `fc`, `phi` arguments instead of pre-mixed IQ); it now uses a persistent per-thread scratch buffer instead of per-call allocation. The tracking path no longer uses it; it remains the acquisition correlator (`sdr_search_code()`).
- `sdr_trk_t` : new member `csk_ref` ; for L6 the `code_fft` member now holds the single M-point extended-code FFT and the int8 `code` bank is also generated.
- `sdr_ch_t` : the L6-only `corr` buffer allocation was removed.

Performance

`sdr_ch_update()` benchmark (`test_sdr_perf`), one L6D channel, average us per 4 ms update, minimum over repeated pinned runs. "Old" is the previous sample-domain decoder, "New" is this update.

x86 (AVX2):

FFT library	12 Msps	16 Msps	24 Msps
pocketfft old	592	933	1406
pocketfft new	144	158	180
FFTW+wisdom old	123	167	442
FFTW+wisdom new	79	91	114

Raspberry Pi 5 (Cortex-A76):

FFT library	12 Msps	16 Msps	24 Msps
pocketfft old	1627	2502	4359
pocketfft new	517	564	653
FFTW+wisdom old	871	1282	2712
FFTW+wisdom new	377	423	518

Eight 24 Msps L6 channels now cost about 4.1 ms per 4 ms cycle on a Pi 5 (26 % of 4 cores, FFTW + wisdom), where the previous decoder could barely run one channel per core. The dependence on the FFT library also shrinks, since only the fixed M = 10800 transform remains in the tracking path.

Verification

- Synthetic CSK unit test (`test_sdr_ch_update_l6_csk`): a noise-dominated L6D/L6E signal (signal amplitude below the noise floor, deterministic LCG noise) with known shifts {0, 1, 2, 3, 5, 37, 128, 254, 255} at $f_s = 12$ MHz and a fractional code offset. It asserts the decoded symbols, in-phase positive P (this catches the polarity-flip bug, verified to fail on the pre-fix code), E/L shape, and the fixed `csk_ref` after an emulated frame sync. Noise dominance matters: an earlier noise-free version of this test could not expose the polarity defect.

- 180 s live IF data (FE 8CH, 24 Msps, 2026-07-05), L6D PRNs 193-202, offline `pocket_trk` runs:

PRN (C/N0)	before fix	after fix + narrowing
196 (45 dB)	2 x lost	no loss, 174 frames
199 (38 dB)	75 s tracked	170 s tracked, 168 frames
200 (37 dB)	4 x lost, 0 frames	no loss, 156 frames

- Acquisition is unchanged (`sdr_search_code()` output verified bit-exact against the previous implementation).

Files Changed

- `src/sdr_ch.c`: `CSK()` (chip-domain correlation, narrowed search), `gen_csk_code_fft()`, L6 branch of `track_sig()`, `trk_new()` / `trk_init()`
- `src/sdr_func.c`: `sdr_bin_csk()`, `sdr_corr_std()` polarity option, `sdr_corr_fft()` fused API and scratch buffer
- `src/pocket_sdr.h`: API declarations, `sdr_trk_t::csk_ref`
- `test/src/test_sdr_ch.c`, `test/src/test_sdr_func.c`, `test/src/test_sdr_perf.c`, `test/utest/makefile`: tests and benchmark
- `doc/api_ref.md`, `doc/algo_desc.md`, `doc/algo_desc_jp.md`: reference updates